

Reviewer Pack — Schatten p-Norm Lower Bound on Disjointness Communication Co...

Entry aa045d952985 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 01:59:16 UTC

Schatten p-Norm Lower Bound on Disjointness Communication Complexity

Entry ID: aa045d952985 Verdict: INCONCLUSIVE Recorded: 2026-05-11 01:59:16 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative L^p Geometry **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

For any $n \geq 1$, the Schatten p -norm of the communication matrix for the DISJOINTNESS problem on n -bit inputs is $\Omega(n)$, where p is the dual exponent satisfying $1/p + 1/q = 1$ for the optimal lower bound.

Rationale (proposer’s reasoning):

The Schatten p -norm captures the operator structure of the communication matrix, which is tightly linked to the required information-theoretic resources in multi-party communication. For DISJOINTNESS, the norm’s growth with n reflects the inherent difficulty of coordinating disjointness checks.

Taxonomy category: COMM_DISJ (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b8364f476d65f1ac

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gram_schmidt(A):
    n, m = len(A), len(A[0])
    Q = [[Fraction(0) for _ in range(m)] for _ in range(n)]
    R = [[Fraction(0) for _ in range(m)] for _ in range(n)]

    for i in range(n):
        v_i = [A[i][j] for j in range(m)]
        for j in range(i):
            r_ij = sum(Q[j][k] * A[i][k] for k in range(m))
            R[j][i] = r_ij
            for k in range(m):
                v_i[k] -= r_ij * Q[j][k]
        norm_v_i = math.sqrt(sum(v_i[j]**2 for j in range(m)))
        if norm_v_i == 0:
            raise ValueError("Vector is linearly dependent")
        R[i][i] = norm_v_i
        for k in range(m):
            Q[i][k] = v_i[k] / norm_v_i

    return Q, R

def singular_value_decomposition(A):
    n, m = len(A), len(A[0])
    U, S, Vt = [[Fraction(0) for _ in range(n)] for _ in range(n)], \
        [[Fraction(0) for _ in range(min(n, m))] for _ in range(n)], \
        [[Fraction(0) for _ in range(m)] for _ in range(m)]

    Q, R = gram_schmidt(A)
    for i in range(n):
        U[i] = Q[i]
    S = [R[i][i] for i in range(min(n, m))]
    Vt = [[Fraction(0) for _ in range(m)] for _ in range(m)]
    for i in range(min(n, m)):
        Vt[i][i] = Fraction(1) / R[i][i]
```

```

    return U, S, Vt

def schatten_p_norm(matrix, p):
    if p == 2:
        return sum(x**2 for row in matrix for x in row)**0.5
    elif p == float('inf'):
        return max(sum(abs(x) for x in row) for row in matrix)
    else:
        raise ValueError("Unsupported p value")

def disjointness_instance(n):
    A = [random.randint(0, 1) for _ in range(n)]
    B = [random.randint(0, 1) for _ in range(n)]
    return A, B

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        for _ in range(5): # Ensure at least 30 instances per seed
            A, B = disjointness_instance(n)
            communication_matrix = [[A[i] * B[j] for j in range(n)] for i in range(n)]
            norm = schatten_p_norm(communication_matrix, p=2) # Using p=2 as an example
            total_metric_value += norm
            instances_tested += 1

    mean_metric_value = total_metric_value / instances_tested
    if mean_metric_value < n_values[-1]:
        conjecture_holds = False
        counterexample = "Schatten p-norm does not scale linearly with n"

    return {
        "metric_name": "Schatten p-Norm",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 8)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)

```

```

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0 support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Schatten p-norm does not scale linearly with n\")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

instances_tested': 30, 'conjecture_holds': False, 'counterexample': 'Schatten p-
norm does not scale linearly with n'}
TRIAL: {'metric_name': 'Schatten p-Norm', 'metric_value': 10.011753727979853, 'instances_tested': 30,
norm does not scale linearly with n'}
TRIAL: {'metric_name': 'Schatten p-Norm', 'metric_value': 9.731746236597688, 'instances_tested': 30,
norm does not scale linearly with n'}
TRIAL: {'metric_name': 'Schatten p-Norm', 'metric_value': 10.011658675891052, 'instances_tested': 30,
norm does not scale linearly with n'}
TRIAL: {'metric_name': 'Schatten p-Norm', 'metric_value': 9.672856354946585, 'instances_tested': 30,
norm does not scale linearly with n'}
TRIAL: {'metric_name': 'Schatten p-Norm', 'metric_value': 8.734210406193428, 'instances_tested': 30,
norm does not scale linearly with n'}
TRIAL: {'metric_name': 'Schatten p-Norm', 'metric_value': 9.586306087041859, 'instances_tested': 30,
norm does not scale linearly with n'}
TRIAL: {'metric_name': 'Schatten p-Norm', 'metric_value': 9.752608783128863, 'instances_tested': 30,
norm does not scale linearly with n'}
TRIAL: {'metric_name': 'Schatten p-Norm', 'metric_value': 9.97251009486534, 'instances_tested': 30,
norm does not scale linearly with n'}
TRIAL: {'metric_name': 'Schatten p-Norm', 'metric_value': 9.914781880120312, 'instances_tested': 30,
norm does not scale linearly with n'}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_be69cc65.p

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with ZeroDivisionError, preventing reliable results. Pre-registered support condition cannot be evaluated without valid data. | next: Fix the test to handle empty results and re-run with sufficient sample size

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109526
2	propose	ollama_remote	qwen3:8b	0	0	88169
3	preregistration	ollama_remote	qwen3:8b	0	0	24016
4	novelty	ollama_remote	qwen3:8b	0	0	20766
5	novelty	ollama_remote	qwen3:8b	0	0	14352
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16325
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12279
8	judge	ollama_remote	qwen3:8b	0	0	16953

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 302386 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/aa045d952985.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/aa045d952985.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/aa045d952985.tar.gz` (if generated)